
CI360-CI560 MOBILE APPLICATION DEVELOPMENT

Mohammed Shah

MS3026

TABLE OF CONTENTS

<i>requirements of the mobile application</i>	3
<i>research</i>	3
Secondary research	3
apple health	3
Water Minder	5
user interface comparison	6
Primary research	7
positive feedback	9
negative feedback	9
<i>project management and planning</i>	10
development timeline	10
<i>design</i>	11
user interface design	11
visual design and branding	11
<i>development of the application</i>	12
registration page	12
login page	15
main page	17
<i>Testing</i>	19
functional testing	19
debugging	20
<i>Summary</i>	20

REQUIREMENTS OF THE MOBILE APPLICATION

in this project, I had created an app that is designed to track each individual's water intake, this is done by allowing users to create secure accounts, log in and track their water intake using the buttons I created on the user interface. The application is designed to be as accessible as possible. I implemented this by adding large buttons so that all users from whatever age group would be able to use it. Listed below are some features that I have implemented in my prototype. The application was built using Android studio with Java for logic and Xml for the user interface design.

- Registration screen with password strength bar,
- Login page that will allow access into the account created,
- Personal greeting at the top once logged in using the users name,
- Different units of measurements for water,
- A simple button that allows users to quickly add a water measurement that is preset by the user
- A progression bar that will congratulate the user on hitting or surpassing their set water intake goal
- A logout button to securely exit the application

RESEARCH

SECONDARY RESEARCH

The start of the project was focused on analysing the competition that have already been tried and tested by millions of users. I looked into both Apple health and Water minder. The aim of this research was to identify key features that would contribute to usability, effective hydration tracking and to implement suitable versions of these features within my prototype.

APPLE HEALTH

Apple health is the native health app that IOS/Apple users have installed on their devices. It comes default with the device and apple encourages users to use their native app. This is done via notifications that would pop up on the users' devices. It allows users to have an all in 1 app which would track metrics such as steps, heart rate (with a compatible device), sleep and hydration.

The core functionalities of apples hydration tracker:

- Manual water intake entry,
- Customisable daily hydration goals,
- Progress visualisation through charts and summaries,
- Data persistence,
- Integration with user profile data such as age and activity level,

Apple has a clean user interface design and focuses on simplicity and consistency without overloading the user with useless features that they wouldn't use.

Within my own water tracker app, I managed to implement features based on the research I had conducted on apple health.

- Custom water intake entry,
- User defined daily hydration goals
- Progress visualisation
- Persistent storage

I decided to implement these core features because I was able to identify that these are the features that the users mainly use and require. The custom water intake field was important as users find it easy to just type the amount they have been drinking. On example I was given was that a classic Coca-Cola can in the UK contains 330ml of fluid. Users find it easy to quickly type 330ml and press a button to insert that data and see their progression bar rise.

The user defined daily hydration goal was another important feature because rather than enforcing a fixed goal, the user is able to set their own daily target. This caters to a wider range of audience as the average person should drink around 2 litres of water a day but an avid gym goer may want to drink more hence why they would want to edit their hydration goal.

Progress visualisation is just a fun clear way to help motivate users to reach their goal. Apple users find this to be important and it works well as they would like to log onto the health app more just to see the bar progress. This gives the user a sense of accomplishment once they hit their defined goal.

Persistent storage allows users to store their data locally which is important as users would like to retain their progress between app sessions.

These implementations mirror apple health's emphasis on user control and minimal friction when logging hydration.

WATER MINDER

Through my research, I had also identified a third-party application that has millions of downloads. Water minder is used by both apple and android users and focuses exclusively on water intake and hydration habits.

Based on Water Minders design and functionality, the following features were implemented in the Water Tracker Application:

- Quick add options
- Goal exceeding support
- Clear progress feedback
- Minimalist interface design

In addition to manual input, the quick add options such as preset buttons allow users to quickly log common drink sizes.

I also decided to implement goal exceeding support so that users are not restricted to their daily target and are able to exceed it. When this occurs, the app will provide feedback showing that the goal has been surpassed.

The application displays both the current intake and the daily target in millilitres ensuring transparency and shows the user how close they are in meeting their goal.

I also decided to design the user interface to be a clean and easy to use. This reduces unnecessary steps when logging water intake which will cater to the less "tech savvy" audience.

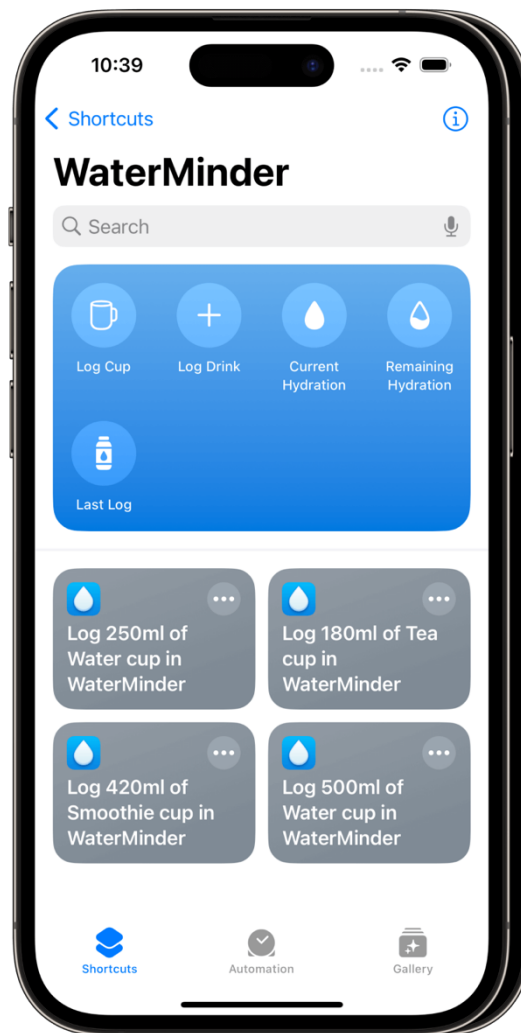
Below shown is a table I have created comparing both apps together against my project

Feature	Apple Health	WaterMinder	Water Tracker (This Project)
Manual water entry	✓	✓	✓
Custom daily target	✓	✓	✓
Progress visualisation	✓	✓	✓
Quick-add buttons	✗	✓	✓

Feature	Apple Health	WaterMinder	Water Tracker (This Project)
Goal exceeding allowed	✓	✓	✓
Persistent data storage	✓	✓	✓
Authentication system	✗	✗	✓

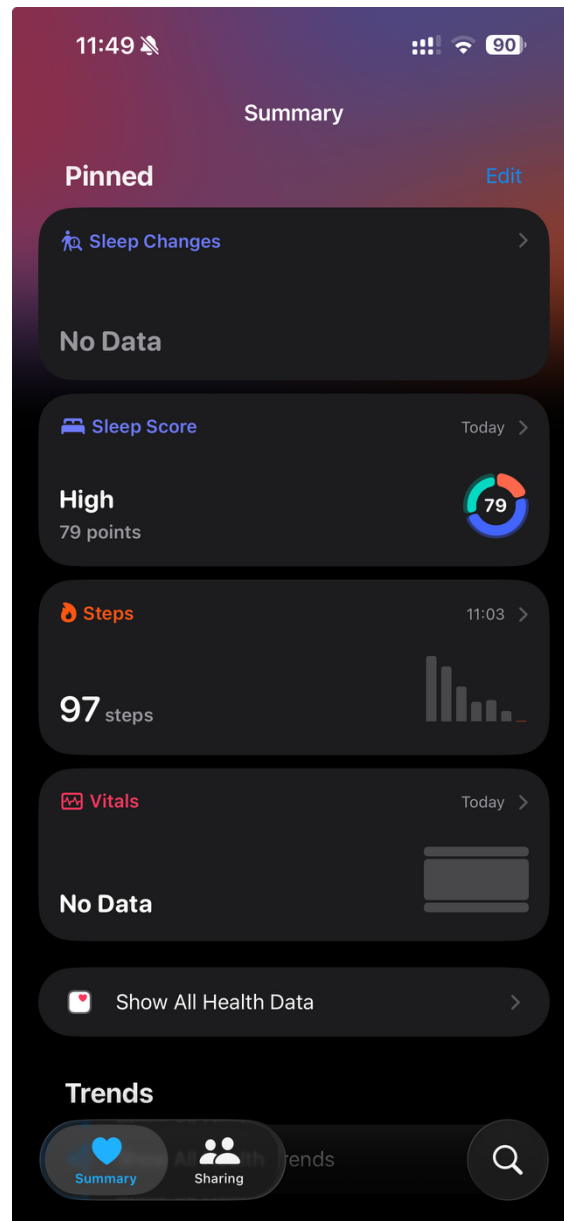
USER INTERFACE COMPARISON

I found that Waterminder's interface is both aesthetically pleasing and user friendly. The buttons within the home screen are colourful and large enough to avoid any mis clicks. Below is a screen shot I have found of Waterminder's user interface.



https://www.google.com/url?sa=t&source=web&rct=j&url=https%3A%2F%2Fblog.waterminder.com%2Ftips%2F20-features-that-will-make-you-fall-in-love-with-the-waterminder-app%2F&ved=0CBYQjRxqFwoTCOi31Zyw_pEDFQAAAAAdAAAAABAI&opi=89978449

apple health has a simple well laid out design that is visually capturing and easy to read. Within the home screen the user can pin their most important categories. This allows the user to personalise its experience and increases returnability and ease of use. Below I have attached a screenshot of my own apple health app.



PRIMARY RESEARCH

I had conducted my own research by creating an anonymous survey and uploading it to my class discussion. I wanted to see what features would be important within the app and what features I should remove. I have attached results of my survey below.

How old are you
3 responses

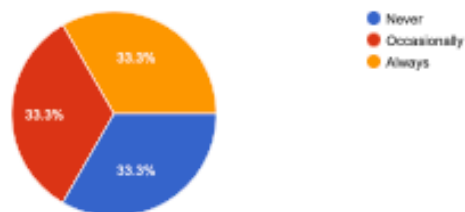
20

21

19

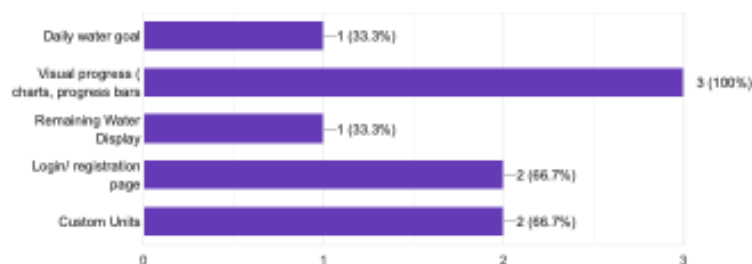
How often do you Track your water intake?
3 responses

[Copy chart](#)



What features would be useful?
3 responses

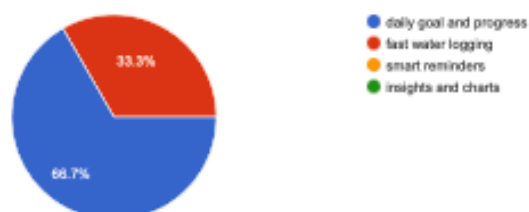
[Copy chart](#)



Which feature is the most important overall?

[Copy chart](#)

3 responses



From the results I had gathered, I have found that users prefer having a visual progress to help motivate them to carry on using the app. They have also reported that the most important feature is the daily goal and progress feature. From these results I was able to implement the features that the users would've wanted.

POSITIVE FEEDBACK

I have also conducted my research of the application after I had developed it and went around and asked peers to review it. Out of the 8 people I had asked, 7 of the reviewers found that the application was smooth from the registration page to the login page to the main screen. Users had an enjoyable experience using my application. Below is a screen shot of my main screen.

The screenshot shows the main interface of the 'WaterTracker' app. At the top left is the app title 'WaterTracker' in a bold, dark font. To its right is a purple rounded button labeled 'Logout'. Below the title is a light purple rounded rectangle containing the text 'Today's intake' followed by '0 / 2500 ml' and a horizontal progress bar that is currently empty. Underneath this is another light purple rounded rectangle for setting the 'Daily target (ml)'. It features a text input field with '2500' entered, a 'Save target' button, and a 'Reset today' button at the bottom. The bottom section is a light purple rounded rectangle for adding water. It has a text input field with 'e.g. 250' as a placeholder, three buttons labeled '+250 ml' and '+500 ml' (along with an 'Add amount' button), and a 'Reset today' button at the bottom.

NEGATIVE FEEDBACK

while the application was widely well received, I had also received some criticism of my developed app.

The main point was that the application looks too plain and not visually appealing. To fix this issue I will add brighter colours and add more onto the home screen.

PROJECT MANAGEMENT AND PLANNING

the development of my hydration tracker was planned to use an incremental approach. I chose this approach as it would allow me to focus section by section rather than overload and implement all the features together. The project was broken down into manageable stages. This allowed me to implement individual features and allow the components within the features to be developed and tested before moving on to a different stage. This method of planning allowed me to refine each feature ensuring its functionality.

DEVELOPMENT TIMELINE

I have created a table which outlined my approximate timeframe for each stage during the whole development of the application alongside the report I've created.

Stage	Description
Week 1	Initial project setup, environment configuration, and review of module requirements
Week 2	Secondary research into existing water tracking applications (Apple Health, WaterMinder)
Week 3	Design planning and wireframe-style layout planning
Week 4	Implementation of Login and Registration activities
Week 5	Implementation of password validation and authentication logic
Week 6	Development of the main water tracking functionality
Week 7	Implementation of daily targets, progress tracking, and goal exceeding logic
Week 8	UI improvements, logo integration, and design refinements
Week 9	Testing, debugging, and error resolution
Week 10	Final testing, signed APK generation, and report completion

DESIGN

the design of this project was guided by the objective of creating a simple and accessible user experience that would mirror the existing hydration trackers that are already being used such as the apple health and WaterMinder. The primary focus was clarity, ease of navigation and minimal user error when inputting data.

USER INTERFACE DESIGN

the application uses a multi activity design, with each screen serving a purpose.

- Login screen for authentication,
- Registration screen for account creation,
- Main screen for hydration tracking and logout,

This separation of screens improves the usability of the application and reduces the cognitive load of users as they aren't presented with unnecessary options at each stage and only see what they need at the present time. This supports a smooth user journey from registration to main screen.

VISUAL DESIGN AND BRANDING

a custom logo was created using generative AI to allow the application to be easily identified. Below is a screenshot of the logo. I had also decided to choose a blue based colour palate as it reflects associations with water and hydration while also maintain sufficient colour contrast for accessibility.



DEVELOPMENT OF THE APPLICATION

For the development of the Water Tracker mobile application, I used java as my coding language of choice. Android studio is the integrated development environment I had used throughout my development. This is because it came preset with all the tools and features, I need to create my application and worked seamlessly with the java code I was writing. It also provides an emulator I can use to view my application in real time as I was developing it.

However, the issues I had encountered while developing my application was that android studio would sometimes crash when running the emulator which caused me to lose progress a couple of times throughout the development of the application which was frustrating. It also sometimes corrupted files which meant I had to clean rebuild the application 3 times.

Using android studio was a simple process when it worked. It allowed me to quickly develop different pages at the same time by just using the drag and drop buttons.

REGISTRATION PAGE

the first page I had created into my application was the registration page. It allows users to create their own account which is then saved for future sessions. This allows users to have their own personalised data every time they log in. I had also created some error conditions such as, incorrect email, invalid name, password not strong enough. Below, I have attached a screenshot of my registration page and a screenshot of the code written to implement this.

```

public class RegisterActivity extends AppCompatActivity {

    4 usages
    private EditText etName, etEmail, etPassword, etConfirmPassword;
    4 usages
    private ProgressBar passwordStrengthBar;
    5 usages
    private TextView tvStrengthLabel, btnGoLogin;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_register);

        etName = findViewById(R.id.etName);
        etEmail = findViewById(R.id.etEmail);
        etPassword = findViewById(R.id.etPassword);
        etConfirmPassword = findViewById(R.id.etConfirmPassword);
        Button btnRegister = findViewById(R.id.btnRegister);
        passwordStrengthBar = findViewById(R.id.passwordStrengthBar);
        tvStrengthLabel = findViewById(R.id.tvStrengthLabel);
        btnGoLogin = findViewById(R.id.btnGoLogin);

        passwordStrengthBar.setMax(100);
    }
}

```



Create your account



Password strength: -

Password must:

- Be at least 8 characters
- Contain at least one special symbol (e.g. ! @ # \$ % ^ &)



Register

[Already have an account? Log in](#)

LOGIN PAGE

the login page works seamlessly with the registration page. Once a user has created an account and clicked register the application will automatically prompt the user to login using the account details they have just created. Below attached is a screenshot of the code and the user interface.

```
1 usage
private void attemptLogin() {
    String email = etEmail.getText().toString().trim();
    String pass = etPassword.getText().toString();

    if (!isValidEmail(email)) {
        etEmail.setError("Enter a valid email");
        etEmail.requestFocus();
        return;
    }
    if (TextUtils.isEmpty(pass)) {
        etPassword.setError("Enter your password");
        etPassword.requestFocus();
        return;
    }

    // Check saved credentials from RegisterActivity
    SharedPreferences sp = getSharedPreferences( name: "auth", MODE_PRIVATE);
    String savedEmail = sp.getString( key: "user_email", defValue: null);
    String savedPass = sp.getString( key: "user_password", defValue: null);

    if (savedEmail != null && savedPass != null &&
        email.equalsIgnoreCase(savedEmail) && pass.equals(savedPass)) {
```



Welcome Back

[No account? Register](#)



MAIN PAGE

Once the user has logged in, the login page would take the user straight to the main page. On the main page there is the progression bar, daily target set button, a logout button in the top right and a button to add your own units of water intake or 2 preset buttons for quick and easy tracking.

WaterTracker

Logout

Today's intake
0 / 2500 ml

Daily target (ml)
2500

Save target

Add water (ml)
e.g. 250

Add amount

+250 ml

+500 ml

Reset today

```

16     public class MainActivity extends AppCompatActivity {
30         protected void onCreate(Bundle savedInstanceState) {
61             currentIntake = sp.getInt(KEY_INTAKE, defValue: 0);
62
63             etTarget.setText(String.valueOf(dailyTarget));
64             updateUI();
65
66             // Save target
67             btnSaveTarget.setOnClickListener( View v -> {
68                 String targetStr = etTarget.getText().toString().trim();
69                 if (TextUtils.isEmpty(targetStr)) {
70                     Toast.makeText( context: this, text: "Enter a target in ml", Toast.LENGTH_SHORT).show();
71                     return;
72                 }
73                 int t;
74                 try {
75                     t = Integer.parseInt(targetStr);
76                 } catch (NumberFormatException e) {
77                     Toast.makeText( context: this, text: "Invalid number", Toast.LENGTH_SHORT).show();
78                     return;
79                 }
80                 if (t <= 0) {
81                     Toast.makeText( context: this, text: "Target must be greater than 0", Toast.LENGTH_SHORT).show();
82                     return;
83                 }
84                 dailyTarget = t;
85                 sp.edit().putInt(KEY_TARGET, dailyTarget).apply();
86                 updateUI();
87                 Toast.makeText( context: this, text: "Daily target saved", Toast.LENGTH_SHORT).show();
88             });
89
90             // Add custom amount
91             btnAddWater.setOnClickListener( View v -> {
92                 String amtStr = etAmount.getText().toString().trim();
93                 if (TextUtils.isEmpty(amtStr)) {
94                     Toast.makeText( context: this, text: "Enter an amount in ml", Toast.LENGTH_SHORT).show();
95                     return;
96                 }

```

TESTING

Testing was carried out throughout the development of the application to ensure that all features functioned correctly and that the user experience was smooth and reliable. Testing was performed using android studio which allowed me to see issues related to the layout, page navigation.

FUNCTIONAL TESTING

functional testing was used to verify that each feature of the application performed as intended and met the requirements that have been outlined. Each core function was manually tested. I done this by using a range of valid and invalid inputs to ensure correct system responses.

I have created a table below highlighting what tests have been carried out.

Test Case	Expected Outcome	Result
Register with valid details	Account created successfully	Pass
Register with weak password	Error message displayed	Pass
Register with mismatched passwords	Registration prevented	Pass
Login with correct credentials	User redirected to main page	Pass
Login with incorrect credentials	Error prompt shown	Pass
Set daily water target	Target saved and displayed	Pass
Add custom water amount	Progress bar updates correctly	Pass
Use preset water buttons	Intake increases by preset amount	Pass
Exceed daily water target	Progress continues past target	Pass
Logout	User returned to login screen	Pass

DEBUGGING

During development, several technical issues were encountered that required debugging. These included XML parsing errors corrupted resource files emulator crashes and Gradle build failures. this was easily done as android studio has built in debugging tools that allowed me to quickly solve most of these issues.

SUMMARY

This project involved the design, development and evaluation of an android-based water tracking application using java and XML. Through secondary research industry standard features were identified and implemented.

The final application includes secure authentication, customisable hydration targets, real time progress tracking and a clean user interface. These features were specifically designed to enhance the user engagement as it provides a streamlined experience of my application.

I faced technical challenges within android studio which were resolved through my own research and experience and tools that android studio had for debugging.

The water tracker application has been designed to focus on a clean design while also maintaining usability and the core functionality of a hydration tracker.

Further development of this application would include more advanced features such as a weekly recap log and push notifications that would remind the user on a set timer to log their activities.